Warehouse Automation

Software Engineering M.Sc. Project

Software Design Description

Authors:

Natai Ella - 208768150
Rodion Novakovskiy - 336121777
A link to this document online

1. Agent Internal Architecture

1.1 Agent Model

Each warehouse agent is modelled as an autonomous cooperative transport agent operating in a partially observable, dynamic, grid-based warehouse environment. The agent's objective is to complete pickup-and-delivery tasks while avoiding collisions, adapting to local changes, and improving its internal knowledge over time.

The system initially includes one main agent type: the Warehouse Transport Agent. Each agent can carry at most one item, may perform one primary physical action per simulation step, and uses communication as a secondary free action when adjacent agents are available.

1.2 Perception Module

The Perception Module is responsible for observing the agent's local environment at the beginning of each simulation step. Since agents do not initially know the full warehouse map, perception is limited to a local radius around the agent.

The module observes:

- Nearby free cells
- Nearby blocked cells
- Nearby agents
- Visible pickup and delivery locations
- Visible item locations
- Whether the intended next cell is occupied
- Local signs of congestion

After perception, the observed information is sent to the Memory Module together with the current simulation timestamp.

1.3 Agent State / Agent Data

Each agent maintains internal state data that describes its own current condition, decision status, and behavioural parameters. This data is separate from the agent's map memory. While memory stores knowledge about the warehouse environment, agent state stores knowledge about the agent itself.

The agent state includes:

- Agent ID
- starting location
- Current position
- Current task ID
- Availability status
- Carrying status
- Carried item ID
- Current target cell
- Current planned route
- Selected intended action for the current tick
- Previous action result
- Waiting counter
- Failed movement counter
- Replanning flag
- Current behavioural mode, such as idle, moving to pickup, moving to delivery, waiting, or replanning

The selected intended action is stored locally before being sent to the Simulation Engine. This allows the system to inspect what the agent planned to do, compare it with what actually happened after validation, and support debugging, replay, and visualization.

1.4 Memory Module

The Memory Module stores the agent's internal knowledge of the warehouse. This includes both information discovered directly through perception and information received from other agents through communication.

The stored memory includes:

- Map size
- Explored map cells
- Known free and blocked cells
- Known pickup and delivery locations
- Known item locations
- Observed agent positions
- Known congestion estimates
- Per-cell historical observations, such as waiting events, failed movement attempts
- Timestamps for all stored information

Each memory entry includes the simulation tick at which the information was last observed or updated. When new information is received, the agent compares timestamps and generally prefers newer information over older information.

The memory module also allows the agent to accumulate local experience over time. For example, if a cell or nearby region repeatedly causes delays, blocked movement, or replanning, the agent may treat that area as more congested or less desirable in future path-planning decisions. This allows the agent to adapt its navigation behaviour using both recent observations and accumulated experience from previous simulation steps.

1.5 Communication Module

The Communication Module handles local peer-to-peer communication with adjacent agents. Communication occurs before the agent finalizes its main action for the current simulation step. This allows the agent to change its movement decision if new information affects its planned path.

When agents communicate, they exchange:

- Known map information
- Timestamps
- Known blocked or free cells
- Item and task information
- Observed agent positions
- Congestion-related information
- Current task
- Current movement target

The receiving agent evaluates each incoming update using:

- Timestamp freshness
- Current stored memory

If the information is accepted, the Memory Module is updated. If the information is rejected, the agent keeps its existing knowledge. An agent may receive multiple information sets from multiple adjacent agents during the same simulation step, and each update is evaluated independently.

1.6 Path Planning Module

The Path Planning Module computes a route from the agent's current position to its current target. The target may be a pickup-adjacent cell, a delivery-adjacent cell, or another intermediate location.

The planner uses the A* algorithm as the base pathfinding mechanism. Route evaluation is based not only on the shortest distance, but also on additional environmental and behavioural costs.

The planner considers:

- Known map information
- Blocked cells
- Occupied or recently occupied cells (by other agents)
- Estimated travel distance
- Congestion estimates
- Historical per-cell failure and waiting observations
- Unexplored or uncertain areas

Each candidate's path is assigned a weighted cost score. Paths with lower estimated cost are generally preferred. For example, shorter, less congested paths incur lower costs, while routes through crowded or previously problematic areas incur higher penalties.

Previous failed routes represent areas where the agent repeatedly experienced blocked movement, excessive waiting, or route replanning. When this occurs, the agent temporarily increases the traversal penalty for those regions, reducing the likelihood of selecting the same route again in the near future. These penalties may gradually decrease over time, allowing previously avoided areas to be reconsidered.

The agent does not always select the single lowest-cost route deterministically. Instead, route selection may include a weighted probabilistic choice between several near-optimal candidate paths. This helps prevent all agents from making identical routing decisions and supports adaptive behaviour in dynamic and partially observable environments.

A path's cost can be calculated like so :

$$Cost(P) = \alpha D(P) + \beta C(P) + \gamma F(P) + \delta U(P)$$

Where:

- P - candidate path
- $D(P)$ - estimated travel distance
- $C(P)$ - congestion estimate along the path
- $F(P)$ - failed-route penalty
- $U(P)$ - uncertainty or unexplored-area penalty
- $\alpha, \beta, \gamma, \delta$ - configurable weights

Lower total cost paths are preferred.

Failed-route penalties increase when the agent repeatedly encounters blocked movement or excessive replanning in the same region.

Congestion values increase in areas with frequent nearby agents or waiting events.

Unexplored areas may optionally receive a small penalty because the agent has less reliable information about them.

Then, when we want to have a probabilistic selection, we can weight the paths like so:

$$P(path_i) = \frac{1/Cost(pathi_i)}{\sum_j 1/Cost(path_j)}$$

Meaning:

Lower-cost paths have a higher probability of selection, but alternative near-optimal routes may occasionally be excluded.

1.7 Congestion Estimation Module

The Congestion Estimation Module allows the agent to gradually learn a local feeling of congestion in different warehouse regions.

Congestion is learned from:

- Repeated observations of nearby agents
- Frequent waiting events
- Failed movement attempts
- Blocked paths

Congestion is not treated as perfect global knowledge. It is a local heuristic estimate that may differ between agents. Over time, if an area repeatedly causes waiting or blocked movement, the agent lowers the probability of choosing routes through that area. If an area becomes clear again, the congestion estimate may gradually decrease.

Formula:

$$C_t(r) = \lambda C_{t-1}(r) + w_a A_t(r) + w_w W_t(r) + w_f F_t(r) + w_b B_t(r) + w_m M_t(r)$$

Where:

- $C_t(r)$ - congestion score of region/cell r at tick t
- λ - decay factor, for example, 0.8 or 0.9
- $A_t(r)$ - number of nearby agents observed in region r
- $W_t(r)$ - waiting events in region r
- $F_t(r)$ - failed movement attempts in the region r
- $B_t(r)$ - blocked-path events in region r
- $M_t(r)$ - congestion information received from other agents
- w_a, w_w, w_f, w_b, w_m - weight values

For path evaluation, a region ***r*** is defined as the cells surrounding a candidate route segment. For example, when comparing two candidate paths, the agent may define ***r*** as all cells on the path together with a small padding area around the path (such as one additional cell in each direction). The agent then evaluates congestion using its remembered observations and learned congestion values for that surrounding region.

This allows the agent to estimate not only whether the exact path is crowded, but also whether nearby areas are likely to create future movement conflicts or bottlenecks.

Congestion may appear to overlap with path-cost factors such as failed movement attempts, waiting, or blocked cells. However, the key difference is that congestion is not measured only on the exact planned path. Instead, congestion is evaluated over a wider region r, which may include the cells on the candidate path as well as nearby cells.

This means that the congestion score represents how crowded or problematic an area is, rather than only how costly a specific path is. For example, two paths may have similar direct movement costs, but one path may pass through a region where many agents were recently observed, where waiting events occurred, or where nearby movement attempts often failed. In that case, the agent may treat that region as more congested even if the exact path itself is still technically valid.

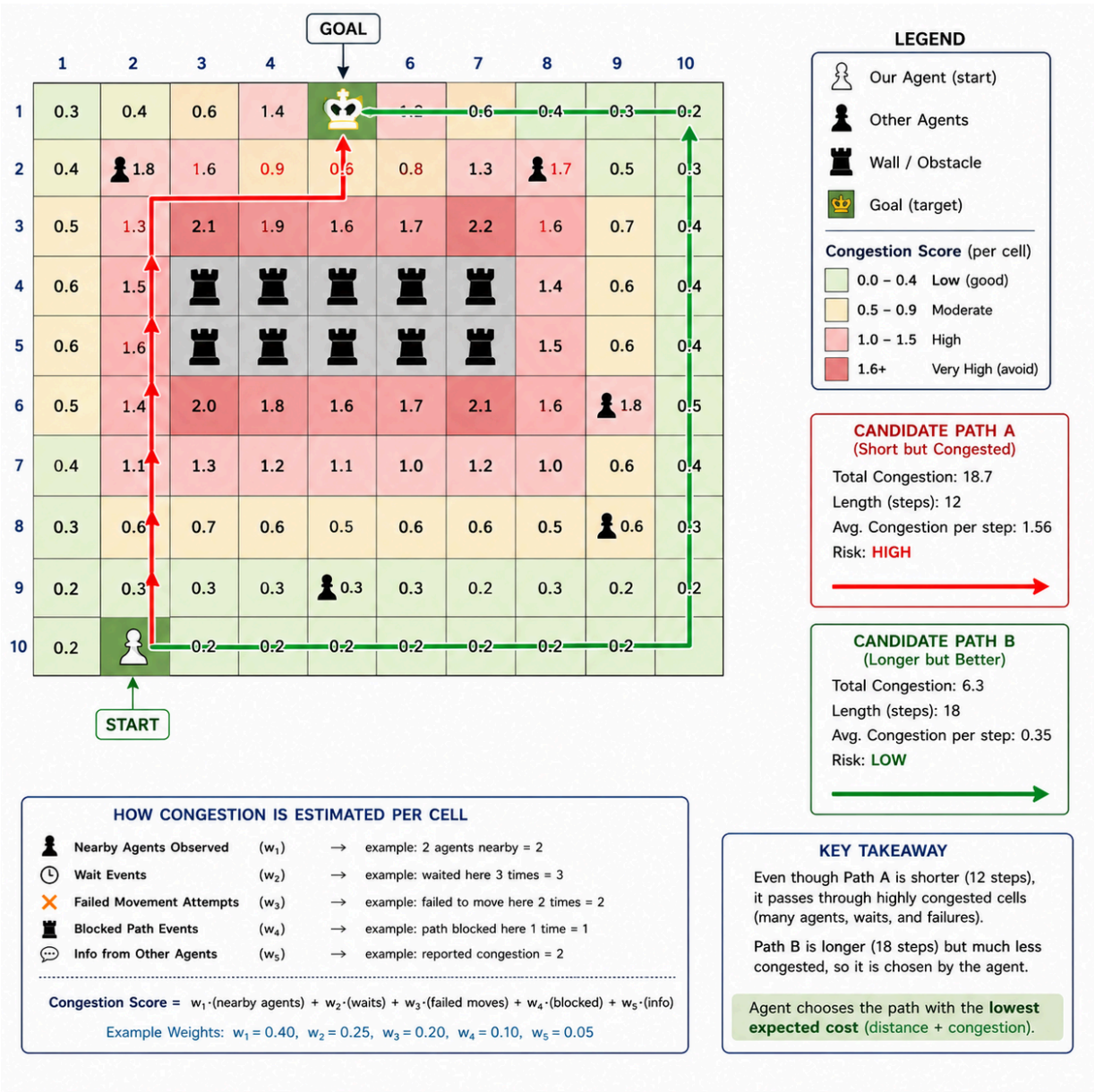
Therefore, the congestion value is used to estimate the general traffic level around a route segment, while the normal path cost evaluates the direct cost of following the path itself.

note:

In this model, the congestion weights are fixed at the start of the simulation. They are manually selected configuration values, not learned values (maybe even different per agent).

1.7.1 Congestion Estimation Example

In this example, the agent needs to reach the goal cell, but it has two possible routes. The red route is shorter, but it passes through a crowded area with other agents, walls, waiting events, and failed movement attempts. As a result, its congestion score is high. The green route is longer but passes through cells with lower congestion, making it safer and more reliable. Therefore, the agent may prefer the longer green path because the expected total cost is lower when congestion is considered, not only distance.



1.8 Decision Module

The Decision Module combines the agent's current task, memory, perception, communication updates, route plan, and congestion estimate to select the agent's intended action for the current simulation step.

Possible intended actions are:

- Move one cell - (N E S W)
- Pick up the item
- Place item
- Wait
- Replan route

The selected action is saved locally as the agent's chosen intended action. The agent does not directly modify the warehouse state. Instead, it submits this intended action to the Simulation Engine, which validates the action against global rules such as occupancy, blocked cells, map boundaries, task validity, and collision conflicts.

1.9 Action Execution Interface

The Action Execution Interface is the boundary between the agent's internal reasoning and the global simulation engine.

The agent outputs an `Action` object, such as:

```
Action(type="MOVE", targetCell=(x,y))
Action(type="PICKUP",EAST, taskId=t)
Action(type="PLACE",WEST, taskId=t)
Action(type="WAIT")
```

The Simulation Engine then determines whether the action succeeds or fails. If the action fails, the agent may update its memory, increase congestion estimation for that area, wait, or trigger replanning during the next simulation step.

1.10 Agent Reasoning Cycle

Each simulation step follows this internal agent cycle:

1. Observe the local environment
2. Update memory with perceived information
3. Communicate with adjacent agents
4. Merge received information
5. Update congestion estimates
6. Check the current task and target
7. Plan or replan the route if needed
8. Select intended action
9. Save chosen action locally
10. Send intended action to the Simulation Engine
11. Receive action result
12. Update internal state based on success or failure

This architecture allows each agent to behave autonomously while still preserving global consistency through the Simulation Engine. The agent reasons locally, but final enforcement remains centralized.

1.11 Agent Log Module

Each agent includes a lightweight local logging interface that reports its internal decisions and observations to the analytics layer. The full structure of local agent logs, global event logs, replay records, and derived metrics is defined in Data & Analytics Design.

2. Interaction & Communication Design

2.1 Communication Model

The warehouse system uses a cooperative peer-to-peer communication model. Agents communicate locally with nearby agents in order to improve their understanding of the environment, reduce congestion, and support safer path planning. The communication system is decentralized, meaning there is no central communication controller or global broadcast mechanism.

Communication is allowed only between agents located in adjacent grid cells. Since the environment is partially observable, agents use communication to gradually expand and synchronize their internal knowledge of the warehouse.

Communication occurs at the beginning of each simulation step, before agents finalize their primary action for that turn. This ordering is important because newly received information may affect route planning, conflict avoidance, or movement decisions. An agent may therefore change its intended movement after receiving updated information from neighbouring agents.

The communication model supports simultaneous local interactions. A single agent may communicate with multiple adjacent agents during the same simulation step and may receive multiple potentially conflicting information updates.

2.2 Communication Protocol

The communication protocol is a lightweight message-based protocol inspired by FIPA-ACL principles. The protocol focuses on local map synchronization and coordination rather than negotiation or voting.

When two adjacent agents communicate, they exchange their currently known information together with the timestamps associated with each information entry.

The communication process follows the sequence below:

1. Agents detect adjacent neighbouring agents.
2. Each agent sends its currently known information to neighbouring agents.
3. Each receiving agent compares incoming information with its own stored knowledge.
4. The receiving agent evaluates:

- timestamp freshness,
- current stored value,

5. The receiving agent either accepts or rejects the update.
6. Accepted information updates the local internal map.
7. After communication completes, the agent selects its action for the current simulation step.

An agent may receive multiple information sets from multiple neighbouring agents during the same simulation step. Each received update shall be evaluated independently.

The protocol uses the following message types:

```
INFORM_MAP_UPDATE
INFORM_CURRENT_TASK
INFORM_CURRENT_TARGET
ACCEPT_UPDATE
REJECT_UPDATE
CONFIRM_UPDATE
```

The protocol supports dynamic replanning and conflict avoidance because agents may share both environmental updates and their current movement intentions before finalizing movement decisions.

2.3 Shared Information

Agents may exchange the following categories of information during communication:

- Known free cells.
- Known blocked cells.
- Pickup locations.
- Delivery locations.
- Item locations.
- Current assigned task. (pick up an item or place an item)
- Current movement target. (not the next move i make but where is my goal)
- Internal route-related observations relevant to navigation.

Each shared information entry shall include:

- the information value itself,
- The timestamp represents when the information was last observed or updated.

When conflicting information is received, agents shall prefer newer information.

This shared-information structure allows agents to gradually build more accurate internal representations of the warehouse while maintaining the environment's partially observable and dynamic nature.

2.4 Sequence Diagram Structures

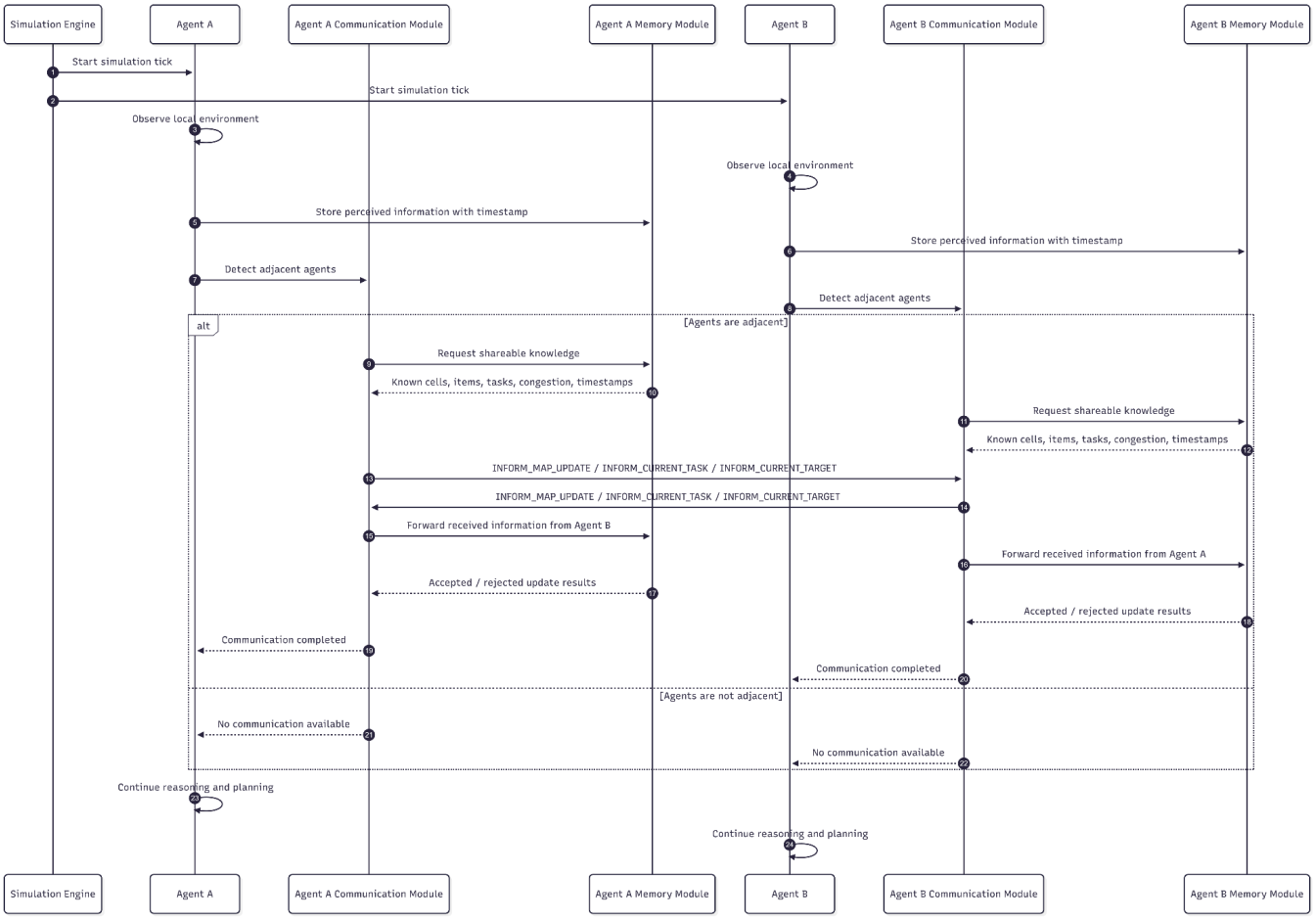
The sequence diagrams in this section describe the interaction flows that are directly relevant to the current warehouse automation design. The system does not use voting or rely on negotiation as a primary coordination mechanism. Auction-based task allocation is considered a possible future extension, but the current design uses local communication, timestamp-based information merging, basic task allocation, and engine-based action validation.

Therefore, the sequence diagrams included in the current SDD are:

1. Communication Protocol
2. Information Merge and Conflict Resolution
3. Task Allocation
4. Collision Avoidance and Replanning

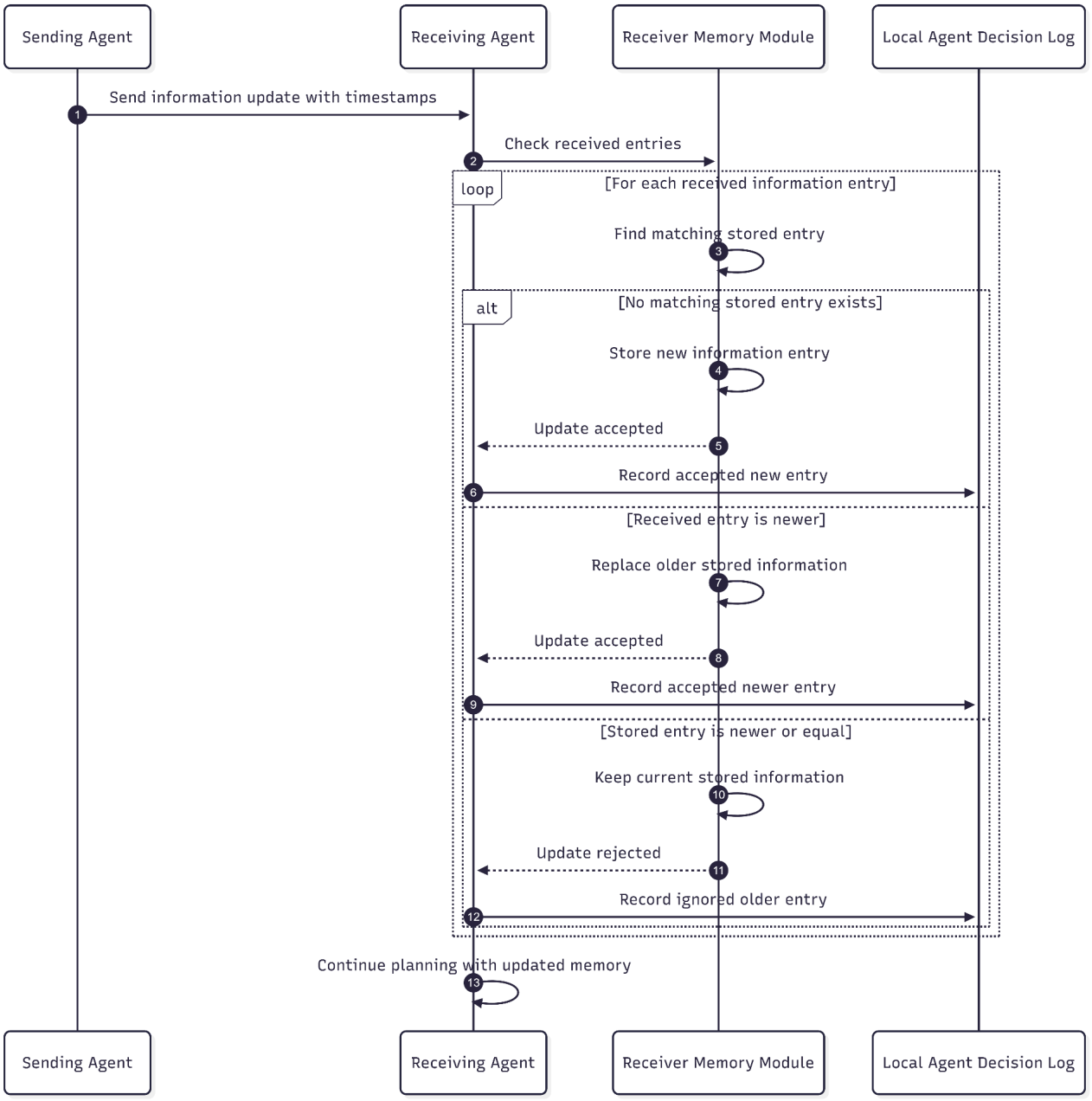
2.4.1 Sequence Diagram 1: Communication Protocol

A higher resolution version of this diagram is available at this link.



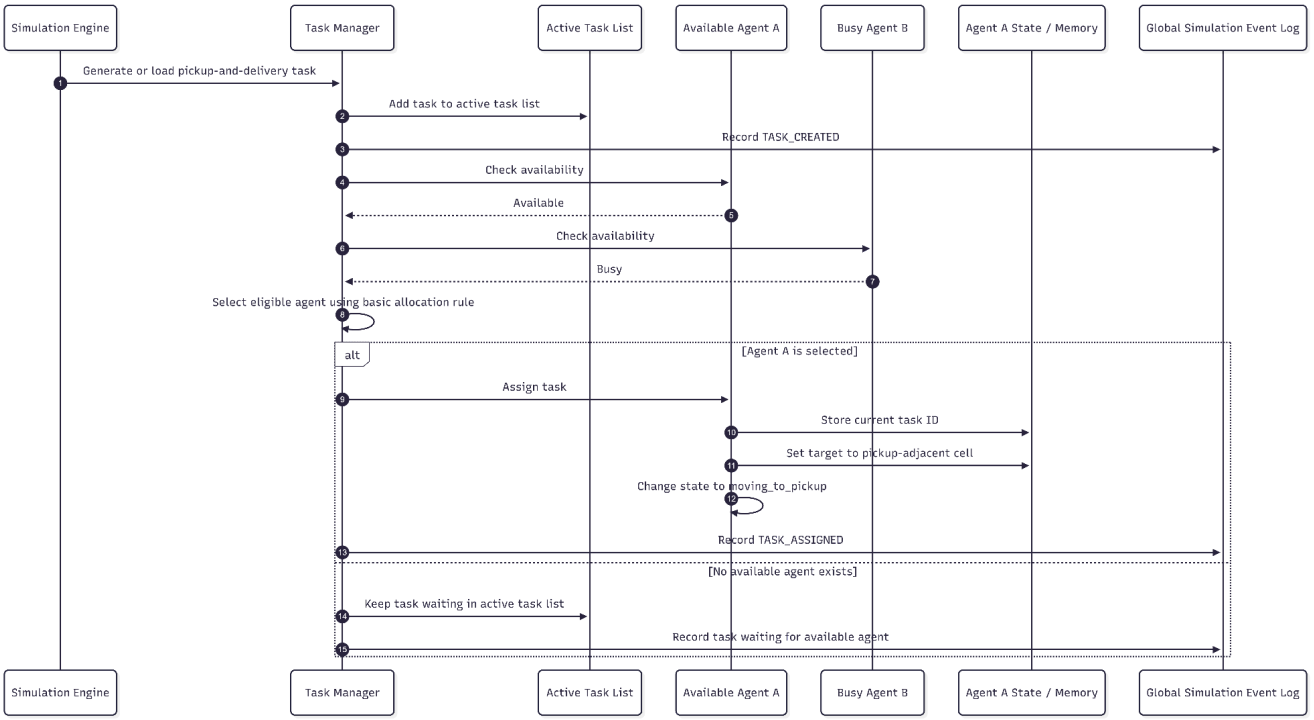
2.4.2 Sequence Diagram 2: Information Merge and Conflict Resolution

A higher resolution version of this diagram is available at this link.



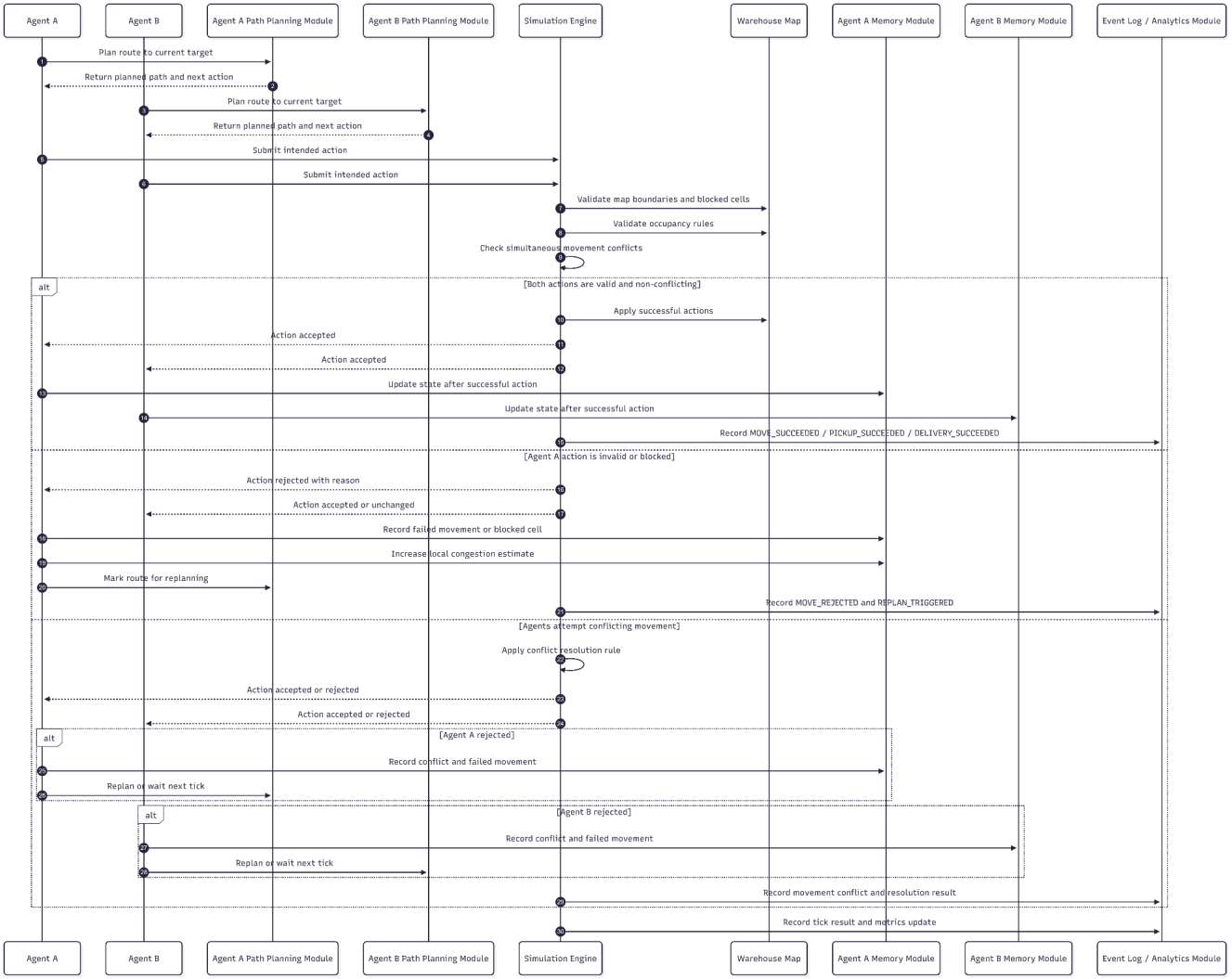
2.4.3 Sequence Diagram 3: Task Allocation

A higher resolution version of this diagram is available at this link.



2.4.4 Sequence Diagram 4: Collision Avoidance and Replanning

A higher resolution version of this diagram is available at this link.



3. Coordination Mechanism Logic

The coordination mechanism defines how multiple warehouse agents cooperate within the same grid-based environment to complete pickup-and-delivery tasks. The system uses a cooperative coordination model: all agents share the same high-level objective of completing warehouse tasks efficiently, safely, and with minimal congestion.

The current design does not rely on voting, negotiation, or auction-based coordination as its primary mechanism. Instead, coordination is achieved through four combined mechanisms:

- Simple task allocation
- Local peer-to-peer information sharing
- Congestion-aware path planning
- Centralized action validation and conflict resolution by the Simulation Engine

This design fits the partially observable and dynamic nature of the warehouse environment. Each agent reasons locally using its own memory, perception, received communication updates, and current task. However, agents do not directly modify the global warehouse state. Final action validation is performed by the Simulation Engine, which preserves global consistency and prevents illegal movement or invalid task execution.

The mechanism guarantees cooperative intent and rule compliance, but it does not guarantee that every individual action will be globally optimal. Since agents operate with limited perception and partial information, conflicts may still occur. The system handles these conflicts by waiting, rejecting movement, replanning, and updating congestion.

3.1 Coordination Overview

Coordination is performed during every simulation tick. A single tick combines local agent reasoning with centralized environment validation.

At a high level, each tick follows this coordination flow:

01.) The Simulation Engine starts a new tick.
02.) Each agent observes its local environment.
03.) Each agent updates its internal memory with perceived information.
04.) Adjacent agents exchange selected local information.
05.) Each receiving agent merges information using timestamp-based conflict resolution.
06.) Each agent updates congestion estimates based on perception, waiting, failed movement, and blocked-path observations.
07.) Idle agents receive or select available tasks.
08.) Each agent plans or replans a route toward its current target.
09.) Each agent selects one intended physical action.
10.) The Simulation Engine collects all intended actions.
11.) The Simulation Engine validates all actions against global constraints.
12.) Valid actions are applied to the warehouse state.
13.) Invalid or conflicting actions are rejected.
14.) Agents receive action results and update their internal state.
15.) Logs and metrics are updated for replay and analysis.

This mechanism allows agents to make autonomous local decisions while the Simulation Engine ensures that the shared environment remains consistent.

3.2 Task Allocation Logic

The initial task allocation mechanism is based on simple agent availability. This follows the current project scope, where auction-based task allocation is treated as a possible future extension rather than the primary implementation.

A task contains:

- Task ID
- Item ID
- Pickup location
- Delivery location
- Current task status
- Assigned agent ID, if assigned

When a new task is created or loaded into the simulation, it is added to the active task list. The Task Manager or Simulation Engine then checks for available agents.

An agent is considered available if:

- It is not currently assigned to another task
- It is not carrying an item
- It is not in a blocked recovery state
- It is able to compute a path or attempt movement toward the pickup area

If one available agent exists, the task may be assigned to that agent. If multiple agents are available, the system may use a simple deterministic rule such as assign the task to the available agent with the lowest estimated path cost to the pickup-adjacent cell.

After assignment, the agent updates its internal state:

- Current task ID is set
- Current target becomes a valid cell adjacent to the pickup location
- Behavioural mode changes to MOVING_TO_PICKUP
- Path planning is triggered

3.3 Coordination Rules and Responsibility Boundaries

The coordination mechanism is distributed between the agents and the Simulation Engine. Agents are responsible for local reasoning and the selection of intended actions, while the Simulation Engine is responsible for maintaining a valid global warehouse state.

The following table summarizes the responsibility boundaries of the coordination mechanism:

Coordination Aspect	Responsible Component	Description
Local perception	Agent	Each agent observes only its local environment and updates its internal memory.
Information exchange	Adjacent agents	Agents share selected local information only with neighbouring agents.
Memory conflict handling	Agent Memory Module	Received information is accepted or ignored according to timestamp-based update rules.
Task allocation	Task Manager / Simulation Engine	Available tasks are assigned to available agents using a simple allocation rule.
Route selection	Agent Path Planning Module	Each agent computes its own path using known map data, congestion estimates, and failed-route history.

Intended action selection	Agent Decision Module	Each agent selects one intended physical action for the current simulation tick.
Global action validation	Simulation Engine	The engine validates all intended actions against the authoritative warehouse state.
Conflict resolution	Simulation Engine	Invalid or conflicting actions are rejected before they can modify the global state.
Recovery after failure	Agent	If an action is rejected, the agent updates its state and may wait, replan, or retry in a later tick.
Analytics and replay	Analytics Layer	Logs and metrics are collected for validation, debugging, and comparison of strategies.

This separation ensures that agents remain autonomous decision-making entities, while the Simulation Engine prevents inconsistent or illegal global states. The agents do not need full global knowledge to cooperate, because their local decisions are filtered through centralized validation before being applied in the warehouse.

The coordination mechanism, therefore, operates as a hybrid model:

- Decentralized reasoning at the agent level
- Local communication between neighbouring agents
- Centralized validation by the Simulation Engine
- Analytics-based verification after each tick

This hybrid structure is suitable for the current project because it preserves partial observability while still allowing the system to prevent illegal moves, collisions, and invalid task execution.

4. System Implementation Design

4.1 Suggested Technical Stack

The system will be implemented using a separate backend-and-frontend architecture. The backend will contain the simulation logic and maintain the warehouse's authoritative state. The frontend will be responsible for visualizing the warehouse state and allowing the user to inspect and control the simulation.

4.1.1 Backend / Simulation Layer

The simulation backend will be implemented in Python using an object-oriented structure. The main simulation components will be represented as classes, including SimulationEngine, WarehouseAgent, WarehouseMap, MapCell, Task, and Item.

The SimulationEngine will manage the global simulation cycle. It will run the tick-based execution loop, update the current simulation step, manage the real warehouse map, and maintain the lists of agents, items, and tasks. It will also be responsible for assigning tasks, validating agent decisions, applying movement, pickup, dropoff, and wait actions, and updating the map after each simulation step.

The backend will expose the current simulation state to the visualization layer. This will allow the frontend to display the warehouse without directly modifying the simulation state.

4.1.2 Backend API

The backend API will be implemented using Flask or FastAPI. The API will allow the frontend dashboard to communicate with the simulation engine.

The API will support operations such as:

- Starting the simulation
- Pausing the simulation
- Resetting the simulation
- Executing a single simulation tick
- Loading predefined scenarios
- Requesting the current warehouse state
- Requesting information about agents, items, and tasks

Communication between the backend and frontend will use JSON. This keeps the data format simple and makes it easy to transfer map state, agent state, item state, task state, and simulation metrics.

4.1.3 Frontend / Visualization Layer

The visualization layer will be implemented as a lightweight web dashboard. The frontend will not directly modify the warehouse state. Instead, it will receive the current state from the backend and render it for the user.

The frontend will be implemented using JavaScript or React. The main visualization will be a top-down 2D warehouse grid. The grid will display agents, items, tasks, blocked cells, pickup locations, delivery locations, and occupied cells.

The dashboard will support:

- Viewing the warehouse map
- Viewing agents and their current positions
- Viewing items and task locations
- Selecting an agent to inspect its current state
- Displaying the selected agent’s current task, carried item, intended action, and planned path
- Starting, pausing, resetting, and stepping through the simulation
- Loading different predefined scenarios
- Displaying basic simulation metrics

The 2D visualization may be implemented using HTML5 Canvas, SVG, or grid-based React components. In a later stage, the system may also support an optional 3D visualization mode using Three.js or React Three Fiber. The 3D mode would use the same backend simulation state, but render the warehouse in a more visual and animated way.

4.1.4 Data and Configuration

The system will use JSON files for scenario definitions and configuration data. Each scenario file may define the warehouse size, blocked cells, initial agent and item positions, pickup and delivery locations, and the initial task list.

JSON will also be used for communication between the backend and frontend. This allows the simulation state to be transferred in a structured and readable format.

Simulation metrics and analysis data may be exported as JSON or CSV. This can support later debugging, replay, and performance comparison between different coordination strategies.

4.1.5 Development and Design Tools

The system will be developed using common lightweight development tools. VS Code or PyCharm will be used for implementation. GitHub will be used for version control and project management. diagrams.net/draw.io will be used for UML class and architecture diagrams. Mermaid may be used for early text-based diagram drafts before manually editing the diagrams in draw.io.

4.2 Main Components

The system is divided into several main components. Each component has a clear responsibility in the simulation and supports the separation between agent reasoning, global simulation control, warehouse state management, and visualization.

4.2.1 Simulation Engine

The Simulation Engine is the central control component of the system. It manages the global tick-based simulation cycle and maintains the warehouse's authoritative state. The engine maintains the real warehouse map, the lists of agents, items, and tasks. During each simulation tick, it collects the agents' selected actions, validates them, applies legal actions, resolves blocked or invalid movements, and updates the warehouse state.

4.2.2 Warehouse Map

The Warehouse Map represents the real grid-based environment. It contains the map cells and stores the current positions of agents and items. The map is updated by the Simulation Engine after movement, pickup, and dropoff actions. It also provides visible-area information to agents during perception.

4.2.3 Warehouse Agents

Warehouse Agents are autonomous entities that operate inside the warehouse. Each agent stores its own current position, current task, carried item, selected action, memory, and planning information. Agents do not directly change the real warehouse map. Instead, they select an intended action, and the Simulation Engine determines whether it is valid.

4.2.4 Agent Internal Modules

Each agent is built from several internal modules. The Perception Module receives local information from the environment, the Memory Module stores the agent's internal knowledge, the Communication Module exchanges information with nearby agents, the Path Planning Module calculates routes, and the Decision Module selects the next action. These modules allow the agent to reason locally while the Simulation Engine preserves global consistency.

4.2.5 Items and Tasks

Items represent objects that must be picked up and delivered. Tasks define the required movement of an item from a pickup location to a delivery location. The Simulation Engine manages the task list and assigns or updates tasks during the simulation. Each agent may have at most one current task and may carry at most one item.

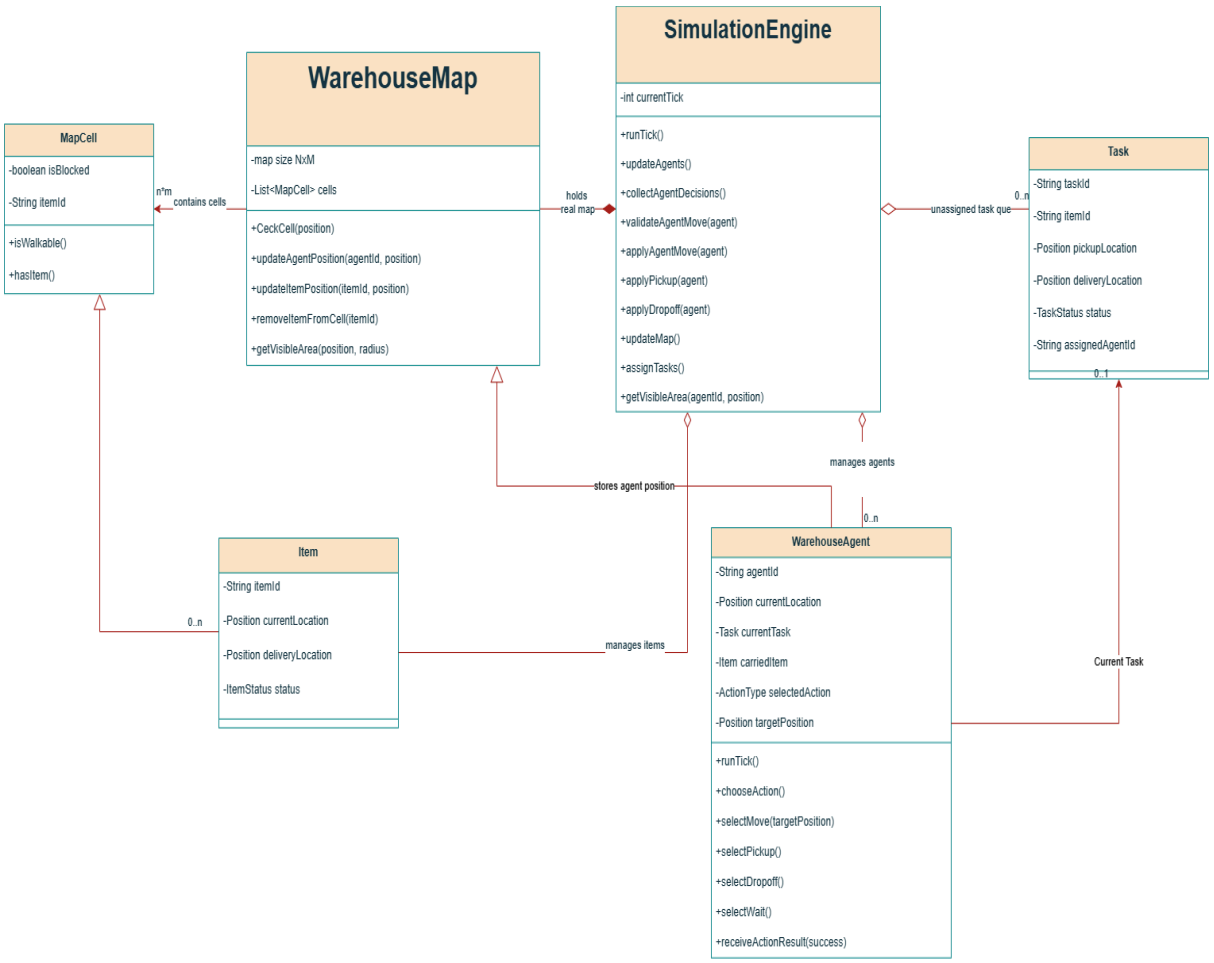
4.2.6 Visualization Dashboard

The Visualization Dashboard displays the current state of the simulation to the user. It shows the warehouse map, agents, items, tasks, blocked cells, pickup locations, delivery locations, and movement information. The dashboard also allows the user to control the simulation by starting, pausing, resetting, or executing a single tick. The visualization layer does not directly modify the simulation state; it only displays information received from the backend.

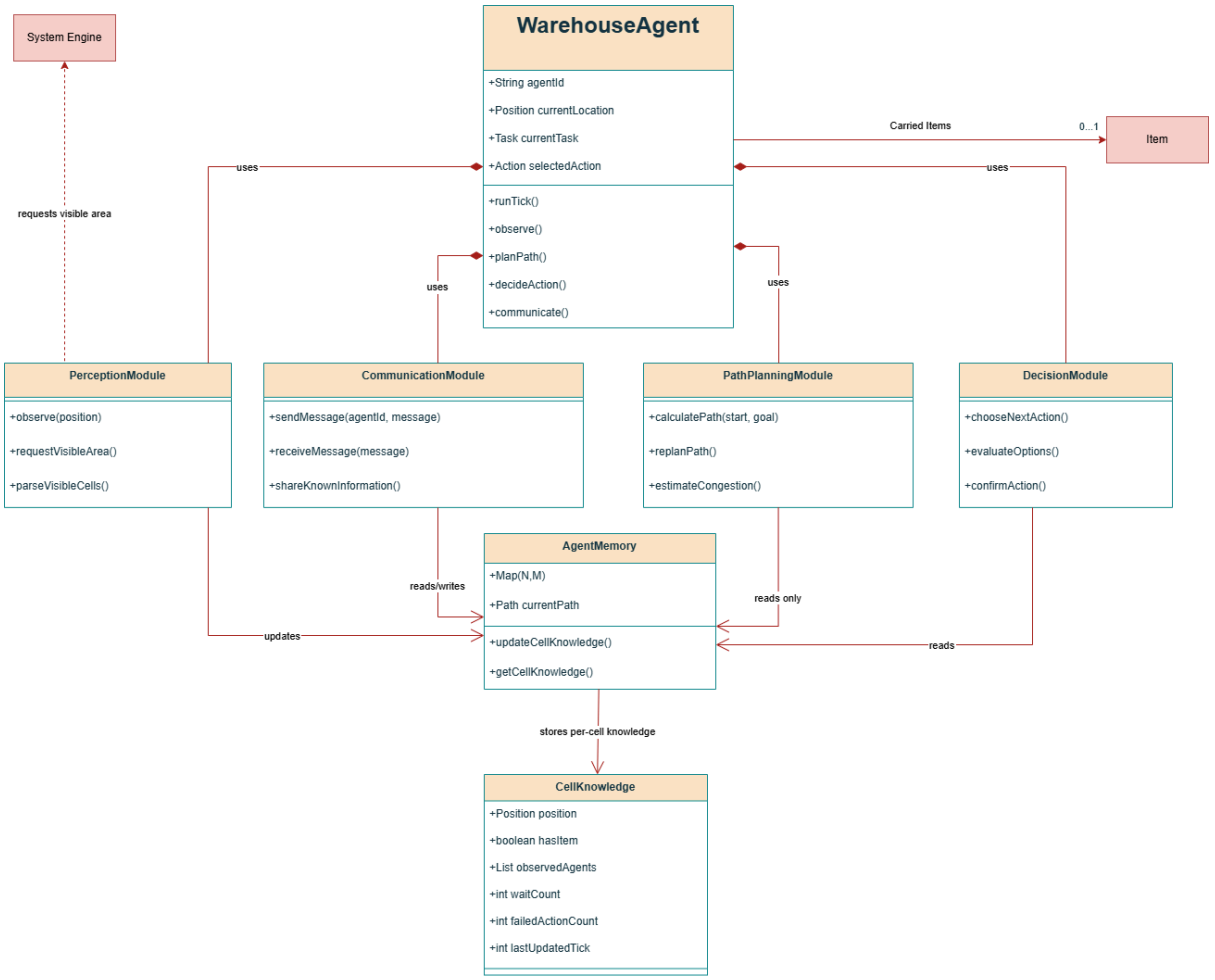
4.2.7 Scenario Configuration

Scenario files define the initial setup of the simulation. A scenario may include the warehouse size, blocked cells, initial agent positions, item positions, pickup and delivery locations, and the initial task list. This allows the system to test different warehouse layouts and coordination situations.

4.3 Class Diagram Structure

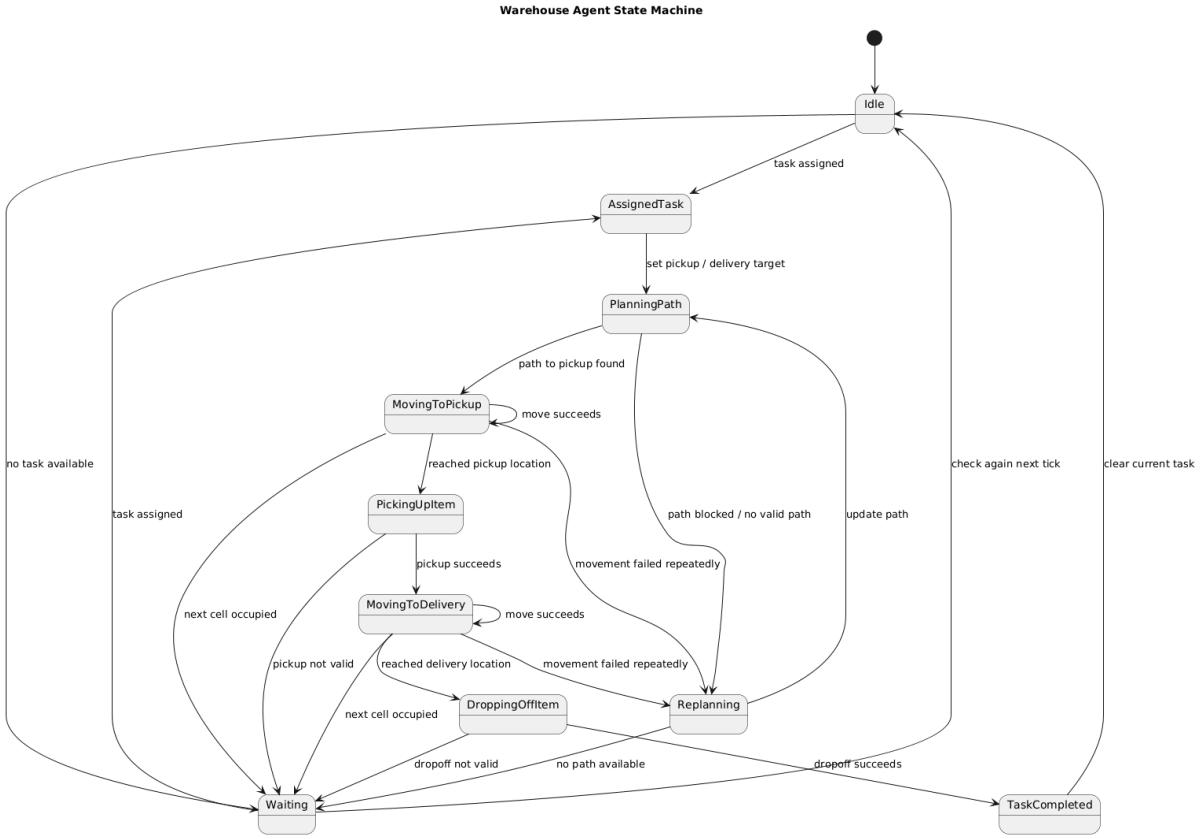


A higher resolution version of this diagram is available at this link.



A higher resolution version of this diagram is available at this link.

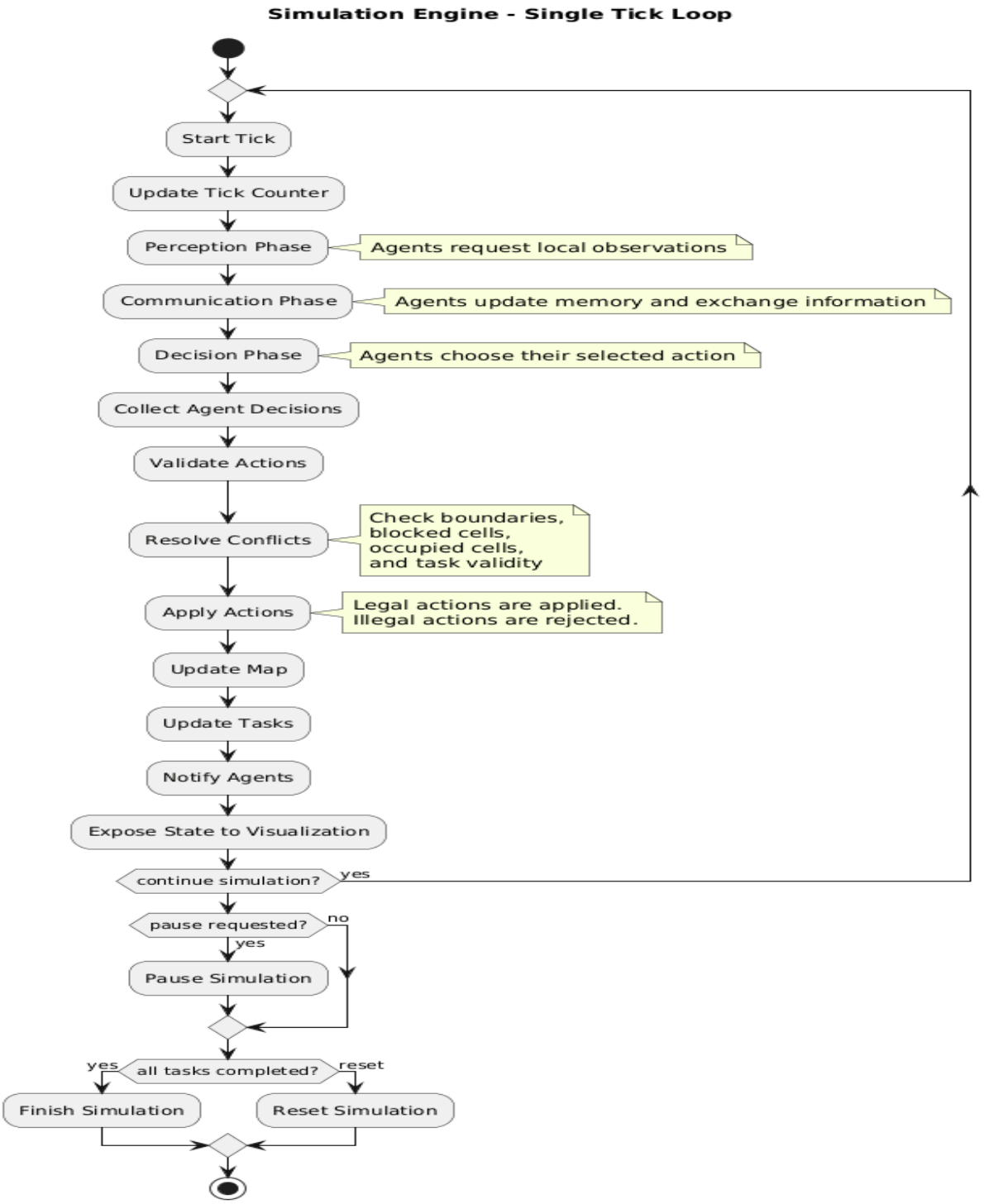
4.4 State Machine Diagram Structure



A higher resolution version of this diagram is available at this link.

4.5 Tick cycle diagrams

A higher resolution version of this diagram is available at this link.



5. Simulator & Visualization Architecture

5.1 Simulation Engine

The simulation engine is designed as the core execution component of the system. It maintains the global warehouse state and advances the simulation using a discrete tick-based execution model.

The engine is responsible for:

- Maintaining the warehouse grid
- Maintaining all agents and active tasks
- Maintaining occupancy information
- Validating agent actions
- Resolving movement conflicts
- Applying successful actions
- Updating task status
- Collecting logs and metrics
- Synchronizing the visualization state

The simulation progresses through a global tick cycle. During each tick:

1. The engine updates the current simulation step.
2. Agents perform local perception.
3. Agents update their internal knowledge.
4. Agents communicate with adjacent agents.
5. Agents merge or reject received information.
6. Agents independently choose intended actions.
7. The engine collects all intended actions.
8. The engine validates actions against global constraints.
9. Conflicting or illegal actions are resolved centrally. (mostly by denying the action and LOGGING it)
10. Successful actions are applied to the warehouse state.
11. Metrics and event logs are updated.
12. The updated state is exposed to the visualization dashboard.

The simulation engine acts as the authoritative controller of the environment. Agents do not directly modify the warehouse state. Instead, they submit intended actions such as movement, pickup, placement, waiting, or communication requests. The engine determines whether those actions succeed based on:

- Map boundaries
- Blocked cells
- Occupancy rules
- Simultaneous movement conflicts
- Task validity conditions

This architecture ensures deterministic simulation behavior and allows the system to support replay, debugging, verification, and future extensions toward more advanced coordination models.

The engine is also designed to support multiple predefined simulation scenarios. Each scenario may define:

- Warehouse dimensions
- Blocked regions
- Pickup and delivery locations
- Initial agent positions
- Initial tasks or order queues
- Obstacle layouts
- Congestion-heavy environments

This allows different coordination strategies and agent behaviours to be tested under controlled conditions.

5.2 Visualization Dashboard

The visualization dashboard is designed as a lightweight observer interface that allows users to inspect and control the simulation in real time.

The primary visualization interface is planned as a simple JavaScript-based top-down 2D grid view of the warehouse. The dashboard renders:

- The warehouse map
- Blocked cells
- Pickup and delivery locations
- Agents
- Items and tasks
- Occupied regions
- Active paths and movement information

The visualization is intended to prioritize clarity and debugging visibility over graphical realism.

5.2.1 Top-Down Map View

The main dashboard view displays the warehouse as a top-down grid. Each cell is visually represented according to its current type and state.

The map visualization should distinguish:

- Free traversal cells
- Blocked cells or shelves
- Pickup locations
- Delivery locations
- Agents
- Agents currently carrying items
- Items waiting for pickup
- Completed delivery areas

Agent movement should update dynamically as the simulation progresses.

5.2.2 Agent Inspection

The dashboard should allow an observer to click on an agent in order to inspect its current internal state and activity.

When selecting an agent, the interface may display:

- Agent identifier
- Current state
- Current task
- Current target
- Current intended action
- Carried item information
- Local memory information
- Movement statistics
- Waiting statistics
- Current planned route/path

The currently planned route may be visualized directly on the grid as a highlighted path extending from the agent's current position toward its current objective.

This functionality is important for validating:

- Route planning behavior
- Congestion handling

- Replanning behavior
- Conflict resolution decisions
- Partially observable reasoning

5.2.3 Task / Object Inspection

The visualization should also support selecting warehouse objects such as tasks, boxes, or delivery items.

When selecting an object, the interface may display:

- Object or task ID
- Pickup location
- Destination location
- Current status
- Assigned or interacting agent
- Creation tick
- Completion information

This allows the observer to track individual deliveries throughout the simulation lifecycle.

5.2.4 Tick and Simulation Controls

The dashboard should include simulation control tools that allow the observer to manage execution flow.

The control interface may include:

- Start simulation
- Pause simulation
- Resume simulation
- Execute a single tick/step
- Reset simulation
- Change simulation speed

The single-tick execution mode is especially important for debugging and validation because it allows observers to inspect agent decisions and movement conflicts step by step.

5.2.5 Scenario Selection

The dashboard should support loading different predefined warehouse scenarios.

Scenario selection may include:

- Different warehouse layouts
- Different starting agent positions
- Different obstacle configurations
- Different congestion levels
- Different task distributions
- Validation and stress-test environments

This functionality supports experimentation and comparison between coordination strategies.

5.2.6 Metrics and Meta Information Panel

The visualization dashboard should include a side panel displaying real-time simulation information and system metrics.

The panel may display:

- Current simulation tick
- Number of completed deliveries
- Active tasks
- Throughput
- Waiting actions
- Blocked movements
- Replanning frequency
- Average task completion time
- Collision prevention statistics
- Active agent count

The dashboard may also display a live event log showing:

- Task creation
- Task completion
- Pickup events
- Failed movement attempts
- Waiting events
- Communication events

- Replanning events

This information supports debugging, validation, and performance analysis of the multi-agent system.

The visualization architecture is intentionally separated from the simulation logic. The frontend dashboard operates only as a rendering and interaction layer and does not directly modify the authoritative simulation state. All simulation updates are received through the backend API layer, which exposes the current state of the warehouse environment and simulation metrics.

In addition to the primary 2D visualization interface, the architecture may later support an optional 3D visualization mode using Three.js and React Three Fiber. The 3D mode would reuse the same backend simulation state and API structure while replacing the rendering layer with a three-dimensional warehouse representation.

In this design, each agent action would correspond to a visual animation sequence. For example:

- Movement actions would trigger walking or translation animations
- Pickup actions would trigger pickup or loading animations
- Placement actions would trigger unloading or delivery animations
- Waiting states could trigger idle animations
- Communication or replanning states could trigger visual indicators or state effects

Because the simulation logic is separated from the visualization layer, the same simulation engine could support both 2D and 3D visualization modes without changing the underlying agent coordination or warehouse logic. The 3D mode is intended primarily as an extended visualization and demonstration layer rather than a separate simulation implementation.

6. Data & Analytics Design

The Data & Analytics Design defines how simulation data is recorded, stored, processed, and used to validate agent strategies. The analytics layer is responsible for collecting event logs, reconstructing the action flow of a simulation, calculating performance metrics, and supporting debugging, replay, and comparison between different coordination strategies.

The analytics design is separated from the agent reasoning logic and from the visualization layer. Agents and the Simulation Engine produce structured events, while the analytics layer stores these events and derives metrics from them. This separation allows the same simulation logic to be evaluated under different scenarios without changing the internal agent implementation.

The main goals of the analytics layer are:

- Record all important simulation events in a structured and replayable format.
- Store both global simulation events and local agent decision data.
- Support step-by-step reconstruction of a complete simulation run.
- Calculate individual agent metrics and global system metrics.
- Validate that agents follow the intended strategies for navigation, task execution, conflict handling, and replanning.
- Compare different scenarios and coordination strategies using consistent measurements.

6.1 Analytics Objectives

The analytics layer supports three main types of analysis.

First, it supports correctness validation. The event logs allow the observer to verify that agents perform only valid actions, do not enter blocked or occupied cells, pick up and place items only under valid conditions, and update their state after each simulation step.

Second, it supports performance evaluation. Metrics such as completion time, throughput, waiting actions, path length, task completion rate, and collision avoidance score allow the system to evaluate how efficiently the agents complete warehouse tasks.

Third, it supports behavioural analysis. Local agent logs and replay data make it possible to inspect why an agent selected a specific action, why it waited, why it replanned, and how congestion or blocked paths affected its decisions.

6.2 Event Log Architecture

The system uses three complementary log types:

- Global Simulation Event Log
- Local Agent Decision Log
- Action Flow Replay Log

Together, these logs provide both the authoritative external view of the simulation and the internal reasoning view of each agent.

6.2.1 Global Simulation Event Log

The Global Simulation Event Log is maintained by the Simulation Engine. It records events that affect the official warehouse state. This log represents the authoritative history of the simulation.

The global event log records:

- Simulation tick advancement
- Task creation
- Task assignment
- Agent movement attempts
- Successful movement events
- Failed movement events
- Pickup events
- Placement / delivery events
- Waiting events
- Route replanning events
- Blocked path events
- Collision or movement conflict prevention
- Task completion events
- Scenario start, pause, reset, and completion events

Each global log entry should include:

- Simulation run ID
- Simulation tick
- Event type
- Agent ID, if relevant
- Task ID, if relevant
- Item ID, if relevant
- Source cell, if relevant
- Target cell, if relevant
- Action result: success, rejected, or ignored
- Rejection reason, if relevant
- Additional event data

The Global Simulation Event Log is used to calculate most system-level metrics, such as completion time, throughput, task completion rate, waiting count, blocked movement count, and collision avoidance score.

6.2.2 Local Agent Decision Log

The Local Agent Decision Log records each agent's internal perspective. Unlike the global event log, which records what officially happened in the environment, the local agent log records what the agent perceived, believed, planned, and attempted to do.

The local agent decision log records:

- Current agent state
- Current task and target
- Perceived nearby cells
- Current planned path
- Selected intended action
- Waiting decision
- Replanning decision
- Failed route observation
- Congestion estimate used for planning
- Communication event with adjacent agents
- Accepted or ignored information updates based on timestamp comparison
- Previous action result received from the Simulation Engine

Each local agent log entry should include:

- Simulation run ID
- Simulation tick
- Agent ID

- Agent state
- Current task ID
- Current target cell
- Selected intended action
- Planned path or next path segment
- Local congestion estimate, if relevant
- Reason for the selected action, if available
- Result of the previous action, if available

This log is especially important for debugging and validation because it allows the observer to compare the agent's intended action with the final action result that the Simulation Engine accepts or rejects.

6.2.3 Action Flow Replay Log

The Action Flow Replay Log stores the complete tick-by-tick flow of the simulation in a replayable format. Its purpose is to support scenario replay, debugging, visual inspection, and post-simulation analysis.

For each simulation tick, the replay log should store:

- Global warehouse state before action execution
- Agent positions before action execution
- Active tasks and task assignments
- Each agent's intended action
- Each agent's planned path or current path segment
- Engine validation result for each action
- Conflict resolution results
- Global warehouse state after action execution
- Metrics snapshot after the tick

This replay structure allows the visualization dashboard to reconstruct the simulation step by step. It also allows an observer to inspect whether an agent's planned path matched its actual movement, whether a blocked movement was caused by another agent, and whether replanning occurred correctly after a failed movement.

6.3 Logged Event Types

The following event types should be supported by the analytics layer:

Event Type	Source	Description	Used For
SIMULATION_STARTED	Engine	A scenario execution begins	Replay, scenario tracking
TICK_STARTED	Engine	A new simulation tick begins	Replay synchronization
TASK_CREATED	Engine	A new pickup-and-delivery task is created	Task completion rate, throughput
TASK_ASSIGNED	Engine / Coordinator	A task is assigned to an agent	Allocation analysis
ACTION_SELECTED	Agent	An agent selects an intended action	Decision validation
MOVE_ATTEMPTED	Engine	An agent attempts movement	Path length, movement attempts
MOVE_SUCCEEDED	Engine	Movement is accepted and applied	Path length, efficiency
MOVE_REJECTED	Engine	Movement is rejected	Blocked movement, collision avoidance
PICKUP_SUCCEEDED	Engine	An item is successfully picked up	Task lifecycle tracking

DELIVERY_SUCCEEDED	Engine	An item is placed at its destination	Completed deliveries, throughput
WAIT_ACTION	Agent / Engine	An agent waits for one tick	Waiting count, congestion analysis
REPLAN_TRIGGERED	Agent	An agent starts route replanning	Replanning frequency
COMMUNICATION_OCCURRED	Agent	Adjacent agents exchange information	Communication analysis
BLOCKED_PATH_DETECTED	Agent / Engine	A path or next cell is blocked	Congestion and failure analysis
TASK_COMPLETED	Engine	A full pickup-and-delivery task is completed	Completion time, TCR
TICK_COMPLETED	Engine	A simulation tick ends	Replay synchronization
SIMULATION_COMPLETED	Engine	The scenario finishes	Final metric calculation

6.4 Metrics Collection

The analytics layer calculates metrics from the event logs after every tick and at the end of each simulation run. Metrics are divided into system-level metrics, agent-level metrics, and path/congestion metrics.

6.4.1 System-Level Metrics

These metrics correspond to the system's required live performance indicators and success measurements.

Metric	Definition	Data Source
Completion Time	Total number of simulation ticks required to complete all tasks	TICK_COMPLETED, SIMULATION_COMPLETED
Completed Deliveries	Number of successfully completed delivery tasks	TASK_COMPLETED
Task Completion Rate	Completed deliveries divided by total tasks	TASK_CREATED, TASK_COMPLETED
Throughput	Completed deliveries divided by simulation time	TASK_COMPLETED, TICK_COMPLETED
Average Task Completion Time	Average number of ticks between task creation and task completion	TASK_CREATED, TASK_COMPLETED
Total Waiting Actions	Total number of wait actions across all agents	WAIT_ACTION
Total Blocked Movements	Number of rejected movement attempts caused by blocked or occupied cells	MOVE_REJECTED
Replanning Frequency	Number of replanning events per simulation run or per agent	REPLAN_TRIGGERED
Collision Avoidance Score	One minus the ratio between collision violations and total movement attempts	MOVE_ATTEMPTED, MOVE_REJECTED
Active Agent Count	Number of agents participating in the simulation	Scenario configuration

6.4.2 Agent-Level Metrics

For each agent, the analytics layer calculates:

Metric	Definition	Purpose
Tasks Completed by Agent	Number of tasks completed by the agent	Measures individual contribution
Total Actions	Number of physical actions selected by the agent	Measures activity level
Useful Actions	Successful movement, pickup, placement, or delivery-related actions	Used for agent efficiency
Agent Efficiency	Useful actions divided by total actions	Measures how much of the agent's activity contributed to progress
Wait Count	Number of wait actions performed by the agent	Measures local congestion or blocking
Failed Movement Count	Number of rejected movement attempts	Measures local path failure
Replanning Count	Number of times the agent recalculated its route	Measures adaptation frequency
Actual Path Length	Number of successful movement steps taken for completed tasks	Used for path inefficiency
Average Delivery Time	Average time required by this agent to complete assigned tasks	Measures individual performance

These metrics allow the system to identify whether performance problems are global or caused by specific agents, regions, or task assignments.

6.4.3 Path and Congestion Metrics

Path and congestion metrics are used to evaluate whether the routing strategy avoids inefficient or crowded routes.

The analytics layer records:

- Actual path length for each completed task
- Shortest feasible path length for each completed task
- Path inefficiency per task
- Total path inefficiency per agent
- Number of blocked movement attempts per region
- Number of waiting events per region
- Number of replanning events caused by congestion
- Repeatedly problematic cells or corridors

For each completed task k:

$$PI_k = L_{actual}(k) - L_{shortest}(k)$$

Where:

- $L_{actual}(k)$ is the actual number of movement steps performed during task k.
- $L_{shortest}(k)$ is the shortest feasible path length between the relevant pickup and delivery route points, calculated using the known scenario map.
- PI_k is the path inefficiency of task k.

For each agent i:

P_i = sum of PI_k over all tasks completed by agent i

This allows the system to compare actual agent movement with the theoretically shorter feasible path and evaluate how much inefficiency was caused by congestion, replanning, partial observability, or conflict avoidance.

6.5 Utility Function and Social Welfare

The system uses an agent utility function to combine several performance aspects into a single score.

For agent i:

$$U_i = \alpha T_i - \beta S_i - \gamma W_i - \delta P_i - \varepsilon C_i$$

Where:

- T_i is the number of tasks completed by agent i.
- S_i is the total number of steps or actions performed by agent i.
- W_i is the number of wait actions performed by agent i.
- P_i is the total path inefficiency of agent i.
- C_i is the number of collision violations or rejected unsafe movement attempts attributed to agent i.
- $\alpha, \beta, \gamma, \delta, \varepsilon$ are positive weight constants.

The utility function rewards task completion and penalizes excessive movement, waiting, inefficient paths, and unsafe behaviour. This makes it useful for comparing different routing strategies, task allocation mechanisms, or congestion-handling policies.

The global social welfare score is calculated as:

$$SW = (\frac{1}{N})\Sigma(U_i)$$

Where N is the number of agents.

A higher social welfare score indicates better collective system performance. This is important because the warehouse system is cooperative: the goal is not to maximize one individual agent's performance, but to improve the overall efficiency and safety of the multi-agent system.

6.6 Replay and Validation Support

The analytics layer supports replay-based validation. A replay file should allow the observer to reconstruct the complete simulation from the initial state to the final state.

Replay support is required for:

- Inspecting why an agent selected a specific action.
- Comparing intended actions with accepted or rejected actions.
- Verifying collision avoidance and movement constraints.
- Analyzing congestion and blocked paths.
- Checking whether replanning occurred after failed movement.
- Comparing performance across different scenarios.
- Demonstrating the correctness of the simulation during evaluation.

For each validation scenario, the analytics layer should produce:

- Final system-level metrics.
- Per-agent performance metrics.
- Event log summary.
- Replay data.
- List of blocked movement and replanning events.
- Task completion summary.

This makes the simulation results measurable, reproducible, and suitable for comparing different agent strategies.